

TinyNPU: Deep Learning Hardware Accelerator

Complete Engineering & Mathematical Reference Manual

Author: Pranav Kamble — Phase 7 Golden Verification Edition

1. Executive Overview & Project Mission

TinyNPU is an end-to-end neural network inference accelerator project designed to bridge high-level Machine Learning (ML) software frameworks and custom Digital Logic design in **SystemVerilog RTL** targeting **FPGAs and ASICs**.

Modern deep learning is developed in Python using floating-point representations (FP32). However, hardware accelerators avoid floating-point operations to minimize silicon area, memory bandwidth, and power consumption. TinyNPU demonstrates the full pipeline:

CSV Dataset → FP32 Training → INT8/INT32 Quantization → .mem Export → SystemVerilog RTL → Golden

2. Network Topology & ML Mathematics

2.1 Architecture Specification

Multilayer Perceptron (MLP) binary classifier configured as **21 → 14 → 7 → 1**:

- **Input Layer:** 21 Features ($\mathbf{x} \in \mathbb{R}^{21}$)
- **Hidden Layer 1:** 14 Neurons + ReLU ($\mathbf{W}^{(1)} \in \mathbb{R}^{14 \times 21}$, $\mathbf{b}^{(1)} \in \mathbb{R}^{14}$)
- **Hidden Layer 2:** 7 Neurons + ReLU ($\mathbf{W}^{(2)} \in \mathbb{R}^{7 \times 14}$, $\mathbf{b}^{(2)} \in \mathbb{R}^7$)
- **Output Layer:** 1 Neuron + Sigmoid ($\mathbf{W}^{(3)} \in \mathbb{R}^{1 \times 7}$, $b^{(3)} \in \mathbb{R}$)

- **Total Parameters:** 399 Weights + 22 Biases = **421 Parameters**

2.2 Forward Propagation Equations

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{h}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = \max(0, \mathbf{z}^{(1)})$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}, \quad \mathbf{h}^{(2)} = \text{ReLU}(\mathbf{z}^{(2)}) = \max(0, \mathbf{z}^{(2)})$$

$$z^{(3)} = \mathbf{W}^{(3)}\mathbf{h}^{(2)} + b^{(3)}, \quad \hat{y} = \sigma(z^{(3)}) = \frac{1}{1 + e^{-z^{(3)}}}$$

2.3 Binary Cross-Entropy Loss & Backpropagation Calculus

$$\mathcal{L}(y, \hat{y}) = -[y \ln(\hat{y} + \epsilon) + (1 - y) \ln(1 - \hat{y} + \epsilon)]$$

Gradient Calculus:

- Output Layer:** $\delta^{(3)} = \frac{\partial \mathcal{L}}{\partial z^{(3)}} = \hat{y} - y$,
 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(3)}} = \delta^{(3)} \mathbf{h}^{(2)T}$,
 $\frac{\partial \mathcal{L}}{\partial b^{(3)}} = \delta^{(3)}$
- Hidden Layer 2:** $\delta^{(2)} = [\mathbf{W}^{(3)T} \delta^{(3)}] \odot \mathbb{I}(\mathbf{z}^{(2)} > 0)$,
 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \delta^{(2)} \mathbf{h}^{(1)T}$,
 $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(2)}} = \delta^{(2)}$
- Hidden Layer 1:** $\delta^{(1)} = [\mathbf{W}^{(2)T} \delta^{(2)}] \odot \mathbb{I}(\mathbf{z}^{(1)} > 0)$,
 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \delta^{(1)} \mathbf{x}^T$,
 $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \delta^{(1)}$
- Gradient Descent Update:** $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$, $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}$ ($\eta = 0.01$)

3. Quantization & Fixed-Point Hardware Arithmetic

3.1 Symmetrical Affine Quantization

$$S = \frac{\max(|X|)}{127}, \quad q = \text{clip} \left(\left\lfloor \frac{x}{S} \right\rfloor, -128, 127 \right)$$

3.2 Why Accumulators & Biases Use INT32

Multiplying two signed INT8 values yields an INT16 product (\$-16384 \dots +16384\$). Accumulating 21 products requires \$16 + \lceil \log_2(21) \rceil = 21\$ bits. Therefore, **32-bit signed accumulators** (`logic signed [31:0]`) prevent overflow.

$$b_{\text{int32}} = \left\lceil \frac{b_{\text{fp32}}}{S_{\text{in}} \cdot S_w} \right\rceil$$

3.3 Dyadic Fixed-Point Requantization

To convert an INT32 accumulator back to an INT8 activation without floating-point division in silicon:

$$M = \frac{S_{\text{in}} \cdot S_w}{S_{\text{out}}} \approx \frac{\text{MULTIPLIER}}{2^{\text{SHIFT}}}, \quad \text{where } \text{SHIFT} = 20, \quad \text{MULTIPLIER} = \lfloor M \cdot 2^{20} \rfloor$$

```
scaled_value = accumulator * MULTIPLIER;    // 32-bit * 32-bit -> 64-bit
shifted_value = scaled_value >>> SHIFT;    // Arithmetic right shift
output_value  = (shifted_value > 127) ? 8'sd127 : (shifted_value < -128) ? -8'sd128 : shifted_value;
```

4. SystemVerilog RTL Architecture

4.1 Module Hierarchy

- `mac.sv` : Multiply-Accumulate unit (\$\text{bias} + \sum x_i w_i\$).
- `relu.sv` : Combinational zero-latency multiplexer (\$(in < 0) ? 0 : in\$).
- `requantize.sv` : Fixed-point scaling, arithmetic shift, and INT8 saturation.

- `neuron.sv` : Complete fused neuron datapath.
- `layer1.sv` : 14 parallel neurons (\$21 \to 14\$, 2352-bit packed weight bus).
- `layer2.sv` : 7 parallel neurons (\$14 \to 7\$, 784-bit packed weight bus).
- `layer3.sv` : 1 output neuron (\$7 \to 1\$, 56-bit packed weight bus).
- `tinynpu.sv` : Top-level integrated neural network engine.

5. Hardware Memory Files & Verification Strategy

Weights, biases, test vectors, and parameters are exported to `data/mem/` in hexadecimal format for standard Verilog `$readmemh()` loading:

- `inputs.mem` : INT8 input samples (2 hex digits per entry).
- `weights{1,2,3}.mem` : INT8 weights.
- `bias{1,2,3}.mem` : INT32 biases (8 hex digits per entry).
- `tinynpu_params.vh` : Auto-generated fixed-point multipliers and shifts.
- `tb_tinynpu_real.sv` : Real-model testbench executing real parameters inside RTL.
- `tools/verify_rtl.py` : Automated golden verification tool comparing Python vs. RTL outputs.

6. Execution Commands

```
# 1. Run full ML training, quantization & memory export
python tools/main.py

# 2. Run golden Python vs. RTL bit-accurate verification
python tools/verify_rtl.py

# 3. Direct Icarus Verilog simulation
cd hardware && make real_sim
```